Comestra

Cozinha, balcão e entrega no mesmo compasso.

Guia completo para instalar, configurar, operar e manter o SaaS de delivery e gestão para restaurantes: API, loja online, back office e app mobile.

API NestJS 11 **PostgreSQL + TypeORM** **Web React 19 + Vite 7** **App Expo SDK 54**
Mercado Pago (cartão e PIX) **Testes Jest + Playwright**

Atualizado em 11/10/2026 · versão da API 0.0.1
Caminhos relativos à raiz do repositório (`backend/` , `frontend-web/` , `frontend-mobile/` , `e2e/` , `docs/`).
Público: desenvolvedores e equipe de implantação. Visão comercial e resumo no `README.md` da raiz.

Sumário

1. Visão geral
2. Checklist de execução rápida
3. Estrutura do repositório
4. Pré-requisitos
5. Variáveis de ambiente
6. Banco de dados PostgreSQL
7. API (backend NestJS)
8. Loja e back office (frontend web)
9. App mobile (Expo)
0. Perfis, usuários padrão e permissões
11. Mapa de endpoints da API
12. Pedidos: regras e transições
13. Modelagem do banco
14. Pagamentos (sandbox e produção)
15. Integrações externas
16. Fluxos principais
17. Uploads e imagens
18. Testes
19. Produção
20. Personalização por instalação
21. Migração a partir do TorresBurgers
22. Limitações conhecidas e roadmap
23. Troubleshooting

1. Visão geral

O **Comestra** é um SaaS de delivery e gestão para restaurantes. Ele cobre o pedido de ponta a ponta, do cardápio online até a entrega, com três aplicações ligadas a uma única API REST: a **loja online** e o **back office** (o mesmo app web, em React) e o **app mobile** (Expo), que serve tanto a clientes quanto à equipe. Cada instalação atende uma loja.

Camada	Tecnologias	Responsabilidade
API (`backend/`)	NestJS 11, TypeScript, TypeORM 0.3, PostgreSQL, Passport JWT, bcrypt, class-validator, Swagger, Sharp, SDK do Mercado Pago	Regras de negócio, autenticação e perfis, preço calculado no servidor, pagamentos, imagens e integrações externas.
Web (`frontend-web/`)	React 19, Vite 7, TailwindCSS 4, React Router 7, Axios, Recharts, Lucide/Iconify, SDK JS v2 do Mercado Pago	Loja (cardápio, carrinho, checkout, Meus Pedidos, Meus Dados) e back office (Dashboard, Pedidos, Cardápio, Equipe, Lixeira).
Mobile (`frontend-mobile/`)	React Native 0.81, Expo SDK 54, NativeWind 2, React Navigation 7, Axios, AsyncStorage	Loja do cliente e painel da equipe por perfil, no mesmo app.
Testes (`e2e/`)	Playwright (API + Google Chrome)	Testes ponta a ponta em ambiente isolado (API :3001, web :5174, banco `comestra_e2e`).

Perfis de acesso

Perfil	Resumo
`ADMIN` (Administrador)	Acesso total: Dashboard, pedidos, cardápio, equipe, lixeira, loja aberta/fechada e manutenção.
`EMPLOYEE` (Atendente)	Conduz todo o fluxo dos pedidos, despacha entregas, cancela e abre/fecha a loja. Não acessa financeiro, cardápio nem equipe.
`KITCHEN` (Cozinha)	Inicia o preparo e marca o pedido como pronto.
`COURIER` (Entregador)	Pega pedidos prontos para entrega e conclui as próprias entregas.
`CLIENT` (Cliente)	Compra pela loja: carrinho, checkout, pagamentos, endereços e acompanhamento dos pedidos.

> As permissões são aplicadas pela API (guards por perfil e regras de transição), e não apenas escondidas nas telas. A identidade do produto (nome, slug `comestra`, domínio `comestra.com.br`) fica em `src/config/brand.ts` de cada app.

2. Checklist de execução rápida

Caminho curto para subir tudo localmente. Os comandos de terminal valem para Linux e para o PowerShell do Windows, exceto onde indicado.

1. Clonar o projeto

```
git clone <URL_DO_REPOSITORIO> comestra
cd comestra
```

2. Criar o banco

```
# Windows (PowerShell) — troque 18 pela versão instalada do PostgreSQL
& "C:\Program Files\PostgreSQL\18\bin\createdb.exe" -U postgres comestra

# Linux
sudo -u postgres createdb comestra
```

3. Configurar e subir a API

```
cd backend
cp .env.example .env        # Windows: Copy-Item .env.example .env
# edite o .env: DB_PASSWORD = senha do usuário postgres
npm install
npm run start:dev
```

A API cria as tabelas, a configuração da loja (aberta) e os cinco usuários padrão. Para conferir: `http://localhost:3000/health` deve responder `{"status":"ok","name":"Comestra","version":"0.0.1"}`, e o Swagger fica em `http://localhost:3000/api`.

4. Subir a loja e o back office

```
cd ../frontend-web
npm install
npm run dev
```

Loja em `http://localhost:5173`, login do cliente em `/signin` e da equipe em `/login`.

5. Rodar o app mobile

```
cd ../frontend-mobile
npm install
npm run start:lan
```

Escaneie o QR Code com o Expo Go (SDK 54), com o celular no mesmo Wi-Fi do computador. Outras formas de conexão estão na seção 9.

6. Entrar com os usuários padrão

Perfil	E-mail	Senha	Onde entrar
Administrador	`admin@comestra.com.br`	`Admin@123`	Web `/login` · App: Entrar → Acesso de Funcionário
Atendente	`atendente@comestra.com.br`	`Atendente@123`	Web `/login` · App: Entrar → Acesso de Funcionário

Perfil	E-mail	Senha	Onde entrar
Cozinha	`cozinha@comestra.com.br`	`Cozinha@123`	Web `/login` · App: Entrar → Acesso de Funcionário
Entregador	`entregador@comestra.com.br`	`Entregador@123`	Web `/login` · App: Entrar → Acesso de Funcionário
Cliente	`cliente@comestra.com.br`	`Cliente@123`	Web `/signin` · App: Entrar → Entrar como Cliente

7. Cadastrar o cardápio

Em um banco novo, o cardápio começa vazio. Entre como Administrador e cadastre produtos e adicionais em **Cardápio** (web).

---

3. Estrutura do repositório

```
comestra/
├── backend/                    API REST (NestJS)
│   ├── src/auth/               login, JWT, RolesGuard, autenticação opcional
│   ├── src/users/              usuários, perfis, avatar e regras de acesso (users.policy.ts)
│   ├── src/products/           produtos, categorias e adicionais
│   ├── src/orders/             orçamento/preço (order-pricing.service.ts), regras puras (order-
rules.ts),
│   │                           transições, dashboard e logística
│   ├── src/payment/            Mercado Pago (cartão e PIX), modo sandbox (payment-rules.ts)
│   ├── src/addresses/          endereços e busca de CEP
│   ├── src/store/              loja aberta/fechada
│   ├── src/promotions/         banner do dia
│   ├── src/common/             upload/otimização de imagens, manutenção, validadores
│   ├── src/config/             brand.ts, env.ts, jwt.ts, app-setup.ts (CORS e validação), paths.ts
│   ├── src/database/           seed dos usuários padrão
│   ├── test/                   teste E2E da API (Jest + Supertest)
│   ├── uploads/                imagens servidas em /uploads
│   └── .env.example
├── frontend-web/               loja + back office (React + Vite)
│   ├── public/favicon.svg      marca
│   └── src/                    pages/ (Client/ e painel), components/, contexts/, layouts/,
│                               routes/, services/, utils/, config/brand.ts
├── frontend-mobile/            app do cliente e da equipe (Expo)
│   ├── assets/                 ícone, splash e favicon
│   ├── src/                    screens/, components/, contexts/, routes/, services/, utils/,
config/brand.ts
│   └── start-tunnel.js         npm run tunnel (Serveo → EXPO_PUBLIC_API_URL no .env.local)
├── e2e/                        testes ponta a ponta (Playwright)
│   ├── support/  scripts/
│   └── tests/                  global.setup.ts, api/identity, api/commerce, web/client, web/admin
└── docs/                       este guia (HTML e PDF)
```

Cada app tem um `.env.example` comentado. O `.env` real é ignorado pelo Git. O repositório não traz `Dockerfile` nem `docker-compose.yml`: o setup documentado aqui usa PostgreSQL instalado localmente e `npm`.

---

4. Pré-requisitos

Ferramenta	Versão	Observações
Node.js e npm	22+ (LTS)	Necessário para os três apps e para a suíte E2E. Cada app usa o próprio `package-lock.json`.
PostgreSQL	14+	Banco padrão: `comestra`. As tabelas são criadas pela API.
Expo Go	Compatível com o SDK 54	Para testar no celular. No emulador Android, o Expo CLI instala a versão compatível.
OpenSSH (`ssh`)	Qualquer recente	Só para o `npm run tunnel` (Serveo).
Google Chrome	Atual	Só para a suíte E2E (o Playwright usa o Chrome instalado).

5. Variáveis de ambiente

API: `backend/.env`

Variáveis definidas no próprio ambiente (servidor, Docker, CI) têm prioridade sobre o `.env`. Valores vazios usam o padrão.

Variável	Padrão	Descrição
`NODE_ENV`	`development`	`production` exige `JWT_SECRET`, desliga o seed dos usuários padrão e o sandbox automático de pagamentos.
`PORT`	`3000`	Porta HTTP. A API escuta em `0.0.0.0` (acessível pela rede local).
`DB_HOST` / `DB_PORT`	`localhost` / `5432`	Servidor PostgreSQL.
`DB_USERNAME` / `DB_PASSWORD`	`postgres` / vazio	Credenciais do banco.
`DB_DATABASE`	`comestra`	Nome do banco (vazio = slug da marca).
`DB_SYNCHRONIZE`	`true`	Cria/ajusta as tabelas a partir das entidades. Produção: `false` + migrations.
`JWT_SECRET`	segredo fixo de desenvolvimento	Assina e verifica os tokens. Obrigatório em produção (sem ele, a API não sobe). Sugestão: 48+ bytes aleatórios.
`JWT_EXPIRES_IN`	`7d`	`7d`, `12h`, `30m` ou segundos. Valor inválido cai no padrão, com aviso.
`APP_NAME`	`Comestra`	Nome no Swagger, em `/health` e na descrição dos pagamentos.
`UPLOADS_DIR`	`./uploads`	Pasta das imagens (servidas em `/uploads`). Produção: volume persistente.
`CORS_ORIGINS`	vazio = qualquer origem	Origens liberadas, separadas por vírgula. O app mobile não envia `Origin` e não é afetado.
`SEED_DEMO_USERS`	ligado fora de produção	Cria os usuários padrão na subida (só os que faltam).
`MP_ACCESS_TOKEN`	vazio	Access token do Mercado Pago. Credenciais de teste começam com `TEST-`.
`PAYMENT_SANDBOX`	automático	`true` / `false`. Vazio = sandbox quando não há token ou ele é `TEST-`, nunca com `NODE_ENV=production`.
`PIX_SANDBOX_APPROVAL_SECONDS`	`5`	No sandbox, segundos até o PIX ser considerado pago.
`FOOTBALL_DATA_TOKEN`	vazio	Token gratuito de football-data.org para o banner de jogos. Sem ele, vale a promoção do dia.

Booleanos aceitam `true/false`, `1/0`, `yes/no`, `sim/não` e `on/off`.

Web: `frontend-web/.env`

Variável	Padrão	Descrição
`VITE_API_URL`	`http://localhost:3000`	Endereço da API, sem barra no final (IP da rede ou URL do túnel para outro aparelho).
`VITE_APP_NAME`	`Comestra`	Nome exibido (logo, título da aba, rodapé).
`VITE_MP_PUBLIC_KEY`	chave de teste	Chave pública do Mercado Pago para tokenizar o cartão. O access token fica só na API.

> Variáveis `VITE_*` entram no JavaScript público do site. Nunca coloque segredos nelas. Depois de mudar o `.env`, reinicie o `npm run dev` (ou gere o build de novo).

Mobile: `frontend-mobile/.env.local`

Variável	Padrão	Descrição
`EXPO_PUBLIC_API_URL`	automático	Força o endereço da API. Vazio usa o IP da máquina do Expo na porta 3000. O `npm run tunnel` preenche esta variável.
`EXPO_PUBLIC_APP_NAME`	`Comestra`	Nome exibido no app.
`EXPO_PUBLIC_MP_PUBLIC_KEY`	chave de teste	Chave pública do Mercado Pago (produção: `APP_USR-...`).

Variáveis `EXPO_PUBLIC_*` entram no bundle. Depois de mudar, reinicie com `npx expo start --clear`.

Testes E2E (`e2e/`)

Variável	Padrão	Descrição
`E2E_API_PORT` / `E2E_WEB_PORT`	`3001` / `5174`	Portas do ambiente de teste.
`E2E_DB_DATABASE`	`comestra_e2e`	Banco de teste (precisa terminar em `_e2e`).
`DB_HOST`, `DB_PORT`, `DB_USERNAME`, `DB_PASSWORD`	`localhost`, `5432`, `postgres`, vazio	Conexão usada para recriar o banco de teste. Informe `DB_PASSWORD` no ambiente se o `postgres` tiver senha.

---

6. Banco de dados PostgreSQL

Configuração TypeORM

A conexão é configurada em `backend/src/app.module.ts`:

```
TypeOrmModule.forRootAsync({
  useFactory: () => ({
    type: 'postgres',
    host: process.env.DB_HOST || 'localhost',
    port: Number(process.env.DB_PORT) || 5432,
    username: process.env.DB_USERNAME || 'postgres',
    password: process.env.DB_PASSWORD ?? '',
    database: process.env.DB_DATABASE || BRAND.slug,      // 'comestra'
    entities: [User, Product, Order, OrderItem, Address, Addon, StoreConfig],
    autoLoadEntities: true,
    synchronize: envFlag('DB_SYNCHRONIZE', true),
  }),
})
```

Produção: o `synchronize` é prático em desenvolvimento, mas pode apagar colunas. Em produção, use `DB_SYNCHRONIZE=false` e versione o schema com migrations. O repositório ainda não traz migrations nem um data source para o CLI do TypeORM: crie-os antes da primeira subida em produção.

Criar um banco limpo

```
# Windows (PowerShell)
& "C:\Program Files\PostgreSQL\18\bin\createdb.exe" -U postgres comestra

# Linux (se o usuário postgres ainda não tiver senha, defina uma: a API conecta via TCP)
sudo -u postgres createdb comestra
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'sua_senha';"
```

Na primeira subida, a API cria as tabelas, a linha única de `store_config` (loja **aberta**) e os usuários padrão. O cardápio começa vazio.

Backup e restauração

```
pg_dump -U postgres -Fc comestra > comestra.dump        # backup
createdb -U postgres comestra_restaurado
pg_restore -U postgres -d comestra_restaurado comestra.dump   # restauração
```

Inclua também a pasta de imagens (`UPLOADS_DIR`) no backup.

Instalações antigas (TorresBurgers)

Para continuar usando o banco antigo, defina `DB_DATABASE=torresburgers` (e `DB_PASSWORD=0000` , se a instalação dependia da senha padrão do código antigo). Para copiar os dados para o nome novo: `createdb -U postgres -T torresburgers comestra` , sem conexões abertas no banco antigo. Com `DB_SYNCHRONIZE=true` , a API acrescenta as colunas novas, como o retrato de preços dos itens.

7. API (backend NestJS)

Comandos

Comando	Finalidade
`npm run start:dev`	Desenvolvimento com recarga automática (porta `PORT`, padrão 3000).
`npm run start:debug`	Desenvolvimento com o inspetor do Node.
`npm run build`	Compila para `dist/`.
`npm run start:prod`	Executa o build (`node dist/main`).
`npm run seed`	Cria/restaura os usuários padrão e redefine nome, perfil e senha.
`npx jest` / `npm test`	Testes unitários (sem banco).
`npm run test:cov`	Testes unitários com cobertura (`coverage/`).
`npm run test:e2e`	Teste E2E da API com Jest + Supertest (precisa de PostgreSQL; use um banco de teste em `DB_DATABASE`).
`npm run lint` / `npm run format`	ESLint (com correção) e Prettier.

URLs importantes

URL	Uso
`http://localhost:3000`	Base da API.
`http://localhost:3000/health`	Saúde: `{ status, name, version }` (para monitoramento e balanceadores).
`http://localhost:3000/api`	Swagger/OpenAPI. Use o botão **Authorize** com o token de `POST /auth/login`.
`http://localhost:3000/uploads/<arquivo>`	Imagens enviadas (pasta `UPLOADS_DIR`).

Comportamento global

- **Validação:** `ValidationPipe` com `whitelist`, `forbidNonWhitelisted` (campos desconhecidos → `400`) e `transform`. A mesma configuração (`src/config/app-setup.ts`) é usada nos testes.
- **CORS:** `CORS_ORIGINS` vazio reflete qualquer origem (desenvolvimento). Com a lista preenchida, libera só essas origens.
- **JWT:** segredo único para assinar e verificar (`src/config/jwt.ts`). Em produção sem `JWT_SECRET`, a API não sobe.
- **Usuário a cada requisição:** a `JwtStrategy` recarrega o usuário do banco. Conta na lixeira → `401`. O perfil usado é sempre o atual.

- **Subida:** cria a configuração da loja (se não existir) e os usuários padrão (se `SEED_DEMO_USERS`). Uma falha no seed não derruba a API.

Módulos

Módulo	Responsabilidade
`AuthModule`	Login, JWT (Passport), `RolesGuard`, `@Roles()` e autenticação opcional (cadastro público).
`UsersModule`	Cadastro de clientes e equipe, edição, avatar, exclusão lógica/definitiva e regras de acesso.
`ProductsModule`	Produtos, categorias, adicionais, disponibilidade e lixeira.
`OrdersModule`	Orçamento e criação (preço no servidor), transições por perfil, cancelamento, despacho, dashboard e logística.
`PaymentModule`	Cartão e PIX (Mercado Pago), sandbox e verificação do pagamento na criação do pedido.
`AddressesModule`	Endereços do cliente e busca de CEP (ViaCEP → BrasilAPI).
`StoreModule`	Loja aberta/fechada e mensagens da vitrine.
`PromotionsModule`	Banner do dia (jogo de futebol ou promoção da semana), com cache em memória.
`CommonModule`	Upload e otimização de imagens e manutenção (conversão em massa para WebP).
`DatabaseModule`	Seed dos usuários padrão na subida.

---

8. Loja e back office (frontend web)

Comandos

```
cd frontend-web
npm install
npm run dev        # http://localhost:5173
npm run build      # verificação de tipos + build em dist/
npm run preview    # serve o build localmente
npm run lint
```

Rotas

Rota	Acesso	Descrição
`/`	Público	Cardápio (busca, categorias, combos, ofertas, banner do dia).
`/cart`	Público (checkout exige login)	Carrinho e checkout.
`/order-success`	Público	Confirmação do pedido.
`/my-orders`	Logado	Meus Pedidos.
`/profile`	Logado	Meus Dados.
`/signin` / `/signup`	Público	Login e cadastro do cliente.

Rota	Acesso	Descrição
`/login`	Público	Login da equipe ("Acessar Sistema").
`/dashboard`	`ADMIN`	Indicadores, gráficos e botão de abrir/fechar a loja.
`/orders`	Equipe	Quadro de pedidos por perfil.
`/products`	`ADMIN`	Cardápio (produtos, adicionais, disponibilidade, lixeira).
`/users`	`ADMIN`	Equipe e usuários.
`/trash`	`ADMIN`	Lixeira de produtos e usuários.
`/admin`	—	Redireciona para `/dashboard`.
outras	Público	Página 404 com o layout da loja.

O Administrador cai em `/dashboard` após o login, e os demais perfis da equipe caem em `/orders`. Uma página fora do perfil redireciona para a página inicial dele. Uma conta da equipe que entra por `/signin` vai para o painel. Um cliente que tenta `/login` fica sem sessão e volta para a loja.

Sessão e estado

- `localStorage`: `comestra.token`, `comestra.user`, `comestra.cart` e `comestra.theme` (`STORAGE_KEYS` em `src/config/brand.ts`).
- O Axios envia `Authorization: Bearer <token>`. Um `401` numa chamada autenticada encerra a sessão e abre o login certo (equipe ou loja).
- Sair remove só a sessão: carrinho e tema continuam salvos.
- Status da loja: consultado a cada 30 s e ao voltar para a aba. Quadro de pedidos: a cada 5 s. Meus Pedidos: a cada 10 s.
- O SDK do Mercado Pago é carregado de forma assíncrona (`src/services/mercadoPago.ts`), sem travar a abertura do site.

---

9. App mobile (Expo)

Comandos

```
cd frontend-mobile
npm install
npm run start:lan     # Expo em modo LAN (mesmo Wi-Fi)
npm run android       # abre no emulador Android
npm run tunnel        # túnel Serveo para a API (grava EXPO_PUBLIC_API_URL no .env.local)
npx tsc --noEmit      # verificação de tipos
```

Outros scripts: `npm start`, `npm run ios` e `npm run web`.

Como o app encontra a API

Ordem de prioridade (`src/services/api.ts`):

1. `EXPO_PUBLIC_API_URL` (`.env` / `.env.local`), preenchida pelo `npm run tunnel`.

2. O IP da máquina que roda o Expo (tirado do endereço do Metro), na porta 3000.
3. `http://localhost:3000` (navegador e simulador iOS).

Situação	Como fazer
Celular no mesmo Wi-Fi	`npm run start:lan`. Automático. Libere a porta 3000 no firewall. No Windows, como administrador: `New-NetFirewallRule -DisplayName "Comestra API" -Direction Inbound -Protocol TCP -LocalPort 3000 -Action Allow`. No Linux com ufw: `sudo ufw allow 3000/tcp`.
Emulador Android	`npm run android` (o modo LAN funciona). Se necessário, use `EXPO_PUBLIC_API_URL=http://10.0.2.2:3000` no `.env.local`, ou `adb reverse tcp:3000 tcp:3000` com `http://127.0.0.1:3000`.
Rede corporativa, 4G ou celular fora da rede	Terminal 1: `npm run tunnel` (deixe aberto). Terminal 2: `npx expo start --clear` (mesma rede) ou `npx expo start --tunnel --clear` (fora da rede). Ao encerrar o túnel (Ctrl+C), o `.env.local` volta ao valor anterior.

Com o Expo em `--tunnel` e sem `EXPO_PUBLIC_API_URL`, o app avisa no log ("Expo em modo túnel: defina EXPO_PUBLIC_API_URL") e cai em `localhost`, que não funciona no celular. Rode o `npm run tunnel` antes.

Navegação por perfil

- Uma única pilha cujas telas dependem da sessão: a equipe vê só o painel (`AdminDashboard`), e clientes e visitantes veem a loja.
- Visitante: na loja, toque em **Entrar** → **Entrar como Cliente** ou **Acesso de Funcionário**. O destino depende do perfil da conta.
- Painel: Administrador com Dashboard, Pedidos, Produtos e Usuários. Atendente com Pedidos ("Gestão de Pedidos"). Cozinha com "Monitor da Cozinha" (Novos e Em preparo). Entregador com "Minhas Entregas" (Prontos para entrega e Minhas entregas, com **Abrir no GPS** e **Ligar**).
- Telas da loja: Home, ProductDetails, Cart, Checkout, AddAddress, PaymentPix, PaymentCard, MyOrders, OrderDetails, Profile ("Minha Conta"), Welcome, SignIn, SignUp e ForgotPassword.

Sessão

O AsyncStorage guarda `@comestra:token`, `@comestra:user` e `@comestra:themeMode`. Um `401` encerra a sessão e abre o login certo, avisando que a sessão expirou. Sair remove só a sessão. O cartão é tokenizado direto na API REST do Mercado Pago com a chave pública: o número não passa pela API do Comestra.

10. Perfis, usuários padrão e permissões

Usuários padrão (um por perfil)

Perfil	E-mail	Senha	O que pode fazer
`ADMIN`	`admin@comestra.com.br`	`Admin@123`	Tudo: Dashboard, quadro completo, despacho, cancelamento, loja, Cardápio, Equipe e Lixeira.
`EMPLOYEE`	`atendente@comestra.com.br`	`Atendente@123`	Quadro completo (aceitar, pronto, despachar, balcão, entrega, cancelar) e abrir/fechar a loja.
`KITCHEN`	`cozinha@comestra.com.br`	`Cozinha@123`	Novos e Preparando: aceitar e marcar como pronto.
`COURIER`	`entregador@comestra.com.br`	`Entregador@123`	Pegar entregas prontas e concluir as próprias entregas.
`CLIENT`	`cliente@comestra.com.br`	`Cliente@123`	Comprar: carrinho, checkout, Meus Pedidos e dados da conta.

- **Na subida da API** (`SEED_DEMO_USERS` , ligado fora de produção): cria só os que faltam e nunca altera contas existentes.
- **`npm run seed`** (pasta `backend`): recria os que faltam, restaura da lixeira e redefine nome, perfil e senha. No fim, mostra uma tabela com perfil, e-mail e senha.
- Fonte: `backend/src/database/seeds/default-users.ts` . Os e-mails usam `BRAND.domain` .

Produção: use `SEED_DEMO_USERS=false` (padrão com `NODE_ENV=production`). Essas senhas são públicas. Se as contas já existirem, exclua-as ou troque as senhas.

Regras de conta

- **Login** (`POST /auth/login`): devolve `{ access_token, user }` . O token traz `sub` , `email` , `role` e `name` e vale 7 dias por padrão.
- **E-mail** gravado sem espaços e em minúsculas. O login não diferencia maiúsculas.
- **Senha:** no mínimo 9 caracteres, com uma letra maiúscula e um número ou símbolo. É guardada com bcrypt (10 rounds) e nunca sai nas respostas.
- **Cadastro** (`POST /users`): sem token, cria `CLIENT` . Outros perfis exigem token de `ADMIN` . Só o `ADMIN` altera o perfil de alguém.
- **Exclusão:** a própria conta ou um `ADMIN` (que não exclui a si mesmo). A exclusão definitiva é só do `ADMIN` e é bloqueada (`409`) para contas com pedidos.
- **Recuperação de senha:** não é automática. Um Administrador define uma nova senha (web: **Equipe**; app: **Usuários** → **editar**).
- **Compras:** pedidos e pagamentos online são exclusivos de `CLIENT` . A equipe recebe `403` ("Somente clientes fazem pedidos e pagamentos...").

11. Mapa de endpoints da API

Base local: `http://localhost:3000`. "Equipe" = `ADMIN`, `EMPLOYEE`, `KITCHEN`, `COURIER`. "Gestão" = `ADMIN`, `EMPLOYEE`.

Saúde e autenticação

Método	Rota	Acesso	Descrição
GET	`/health`	Público	Status, nome e versão.
POST	`/auth/login`	Público	Login por e-mail e senha.

Usuários

Método	Rota	Acesso	Descrição
POST	`/users`	Público (`CLIENT`) · `ADMIN`	Cadastro. JSON ou multipart com o avatar no campo `file`.
GET	`/users`	`ADMIN`	Usuários ativos.
GET	`/users/trash`	`ADMIN`	Usuários na lixeira.
GET	`/users/:id`	Próprio ou `ADMIN`	Detalhe.
PATCH	`/users/:id`	Próprio ou `ADMIN`	Edição (só o `ADMIN` muda `role`), com avatar opcional.
PATCH	`/users/:id/restore`	`ADMIN`	Restaura da lixeira.
DELETE	`/users/:id`	Própria conta ou `ADMIN`	Exclusão lógica.
DELETE	`/users/:id/permanent`	`ADMIN`	Exclusão definitiva (`409` se houver pedidos).

Cardápio

Método	Rota	Acesso	Descrição
GET	`/products`	Público	Produtos fora da lixeira (inclusive indisponíveis), com adicionais permitidos.
GET	`/products/:id`	Público	Detalhe.
GET	`/products/trash`	`ADMIN`	Produtos na lixeira.
POST	`/products`	`ADMIN`	Cadastro (multipart com a foto em `file`, ou JSON).
PATCH	`/products/:id`	`ADMIN`	Edição (foto opcional).
PATCH	`/products/:id/restore`	`ADMIN`	Restaura da lixeira.

Método	Rota	Acesso	Descrição
DELETE	`/products/:id`	`ADMIN`	Move para a lixeira.
DELETE	`/products/:id/permanent`	`ADMIN`	Exclusão definitiva (`409` se o produto aparece em pedidos).
GET	`/addons`	Público	Adicionais fora da lixeira.
POST/PATCH/DELETE	`/addons` , `/addons/:id`	`ADMIN`	Cadastro, edição e exclusão lógica de adicionais.

Pedidos (todas exigem login)

Método	Rota	Acesso	Descrição
POST	`/orders/quote`	`CLIENT`	Orçamento: itens, subtotal, `delivery_fee` , total, tempo estimado e `store_open` .
POST	`/orders`	`CLIENT`	Cria o pedido (com `paymentId` quando pago online).
GET	`/orders/my-orders`	Logado	Pedidos do próprio usuário.
GET	`/orders/:id`	Dono ou equipe	Detalhe (outro cliente → `403`).
PATCH	`/orders/:id/cancel`	Dono (`PENDING`) ou gestão	Cancela.
GET	`/orders`	Equipe	Todos os pedidos.
PATCH	`/orders/:id`	Equipe	Muda o status (`{ status }`), conforme o perfil.
PATCH	`/orders/:id/assign`	Gestão	Atribui o entregador (`{ driverId }`) e leva para `DELIVERING` .
GET	`/orders/:id/nearby`	Gestão	Pedidos prontos, sem entregador, do mesmo bairro.
GET	`/orders/drivers/list`	Gestão	Entregadores ativos.
GET	`/orders/summary`	`ADMIN`	Resumo do dashboard.
GET	`/orders/charts`	`ADMIN`	Faturamento dos últimos 7 dias e os 5 produtos mais vendidos.

Pagamentos, endereços, loja, banner e imagens

Método	Rota	Acesso	Descrição
POST	`/payment/card`	`CLIENT`	Cobra o cartão (token do Mercado Pago). O valor é calculado a partir de `order` .
POST	`/payment/pix`	`CLIENT`	Gera o PIX (QR Code e copia e cola).

Método	Rota	Acesso	Descrição
GET	`/payment/status/:id`	Pagador ou equipe	Status do pagamento (consultado pelas telas de PIX e cartão).
POST	`/payment/webhook`	Público	Só confirma o recebimento (`{ "status": "ok" }`).
GET	`/addresses/cep/:cep`	Público	Busca de CEP (ViaCEP, com BrasilAPI como reserva).
POST/GET	`/addresses`	Logado	Cria / lista os endereços do próprio usuário.
GET/PATCH/DELETE	`/addresses/:id`	Logado (dono)	Detalhe, edição e exclusão lógica (endereço de outra pessoa → `404`).
GET	`/store/status`	Público	`{ id, is_open, opening_message, closing_message }` .
PATCH	`/store/status`	`ADMIN` , `EMPLOYEE`	Abre/fecha a loja e edita as mensagens (até 200 caracteres).
GET	`/promotions/banner`	Público	Banner do dia (`/promotions/football` é um alias antigo).
POST	`/uploads`	`ADMIN`	Imagem avulsa (campo `file`) → `{ filename }` .
POST	`/maintenance/optimize-images`	`ADMIN`	Converte para WebP as imagens antigas de produtos e avatares.

12. Pedidos: regras e transições

As regras ficam em funções puras em `backend/src/orders/order-rules.ts`, cobertas por testes unitários.

Preço calculado no servidor

```
POST /orders
{
  "type": "DELIVERY",
  "addressId": 3,
  "items": [
    { "productId": 1, "quantity": 2, "addonIds": [4, 4], "meatPoint": "Ao Ponto",
      "removedIngredients": ["Cebola"], "observation": "Capricha no molho" }
  ],
  "paymentMethod": "MONEY",
  "changeFor": "R$ 100,00"
}
```

- O cliente nunca envia preços. O preço unitário é o promocional quando ele existe e é menor que o normal. Os cálculos são feitos em centavos.
- `addonIds` repetidos indicam a quantidade (`[4, 4]` = 2×). Se o produto define adicionais permitidos, só eles são aceitos.
- Produtos e adicionais indisponíveis ou na lixeira são recusados com a mensagem do motivo.
- Limites: 50 linhas por pedido, 50 unidades por item, 50 adicionais por item e observação de até 300 caracteres.
- `DELIVERY` exige um endereço ativo do próprio cliente ("Endereço obrigatório para delivery.").
- **Retrato do preço:** cada item grava `unit_price` e `addons_snapshot` (nome, preço e quantidade cobrados). Pedidos antigos sem retrato continuam legíveis.
- **Loja fechada:** o orçamento devolve `store_open: false`, e o pedido e os pagamentos respondem `400` ("A loja está fechada no momento.").

Entrega por bairro

Vale a primeira faixa cujo termo aparece no nome do bairro, sem diferenciar acentos e maiúsculas. Por exemplo, "Barra da Tijuca" entra na faixa da Barra.

Bairro contém	Taxa	Tempo
centro, lapa, santa teresa	R$ 5	30-40 min
flamengo, botafogo, laranjeiras	R$ 7	40-50 min
copacabana, ipanema, leblon	R$ 10	50-60 min
barra, recreio	R$ 20	60-80 min
tijuca, maracana	R$ 12	50-60 min
demais bairros	R$ 15	50-60 min
retirada (`TAKEOUT`)	grátis	15-20 min

A tabela vem de uma instalação no Rio de Janeiro. Ajuste `DELIVERY_ZONES` por instalação.

Pagamento e troco

- Formas: `PIX`, `CREDIT_CARD`, `DEBIT_CARD` e `MONEY`. Com `paymentId`, o pedido foi pago online. Sem ele, o pagamento é feito na entrega ou na retirada (cartão = maquininha).
- O troco só vale para `MONEY`. É gravado como `"R$ 50,00"` e precisa ser maior ou igual ao total.
- Ao concluir (`DONE`) um pedido pago na entrega ou na retirada, `payment_status` vira `PAID`.

Transições de status por perfil

Ação	Gestão	Cozinha	Entregador
Aceitar: `PENDING → PREPARING`	sim	sim	—
Pronto: `PREPARING → READY_FOR_PICKUP`	sim	sim	—
Despachar com entregador escolhido (`/assign`)	sim	—	—
Pegar para entrega: `READY_FOR_PICKUP → DELIVERING`	sim	—	sim (assume se sem responsável)
Confirmar entrega: `DELIVERING → DONE`	sim	—	só as próprias
Balcão (retirada): `READY_FOR_PICKUP → DONE`	sim	—	—
Cancelar	até concluir	—	—

- A gestão (`ADMIN`, `EMPLOYEE`) pode fazer qualquer transição válida. `DONE` e `CANCELED` são finais, o mesmo status é inválido e `TAKEOUT` nunca entra em `DELIVERING`.
- Transição impossível → `400` "Transição inválida: X → Y". Transição possível, mas não permitida ao perfil → `403`.
- O **cliente** cancela o próprio pedido enquanto ele está `PENDING` (`PATCH /orders/:id/cancel`).
- **Despacho** (`/assign`): só para `DELIVERY` em `PREPARING`, `READY_FOR_PICKUP` ou `DELIVERING` (troca de entregador), com um `COURIER` ativo.
- **Concorrência:** a gravação só acontece se o pedido ainda estiver no status lido (e, ao assumir uma entrega, sem entregador). Caso contrário, a resposta é `409`, e só um de dois entregadores simultâneos assume o pedido.
- **Dashboard:** o faturamento conta apenas pedidos `PAID` e não cancelados.

---

13. Modelagem do banco

Enums

Enum	Valores
`UserRole`	`CLIENT`, `ADMIN`, `KITCHEN`, `COURIER`, `EMPLOYEE`
`ProductCategory`	`hamburgueres`, `entradas`, `acompanhamentos`, `bebidas`, `sobremesas`, `molhos`, `adicionais`, `combos`

Enum	Valores
`OrderStatus`	`PENDING`, `PREPARING`, `READY_FOR_PICKUP`, `DELIVERING`, `DONE`, `CANCELED`
`OrderType`	`DELIVERY`, `TAKEOUT`
`PaymentStatus`	`PENDING`, `PAID`, `FAILED`

Tabelas

Tabela	Campos principais	Relações
`users`	`id`, `name`, `email` (único, gravado em minúsculas), `password_hash` (nunca selecionado por padrão), `phone`, `avatar`, `role`, timestamps, `deleted_at`	1:N com `orders` e `addresses`. Também é o entregador (`driver_id`) em pedidos.
`products`	`id`, `name`, `description`, `price`, `promotion_price`, `is_highlight`, `image`, `category`, `ingredients`, `available`, timestamps, `deleted_at`	N:N com `addons` via `product_allowed_addons`. 1:N com `order_items`.
`addons`	`id`, `name`, `description`, `price`, `image`, `available`, `category` (padrão `geral`), `deleted_at`	N:N com produtos e com itens de pedido (`order_item_addons`).
`orders`	`id`, `status`, `payment_status`, `type`, `total_price`, `delivery_fee`, `payment_method`, `change_for`, `payment_id` (único), `estimated_delivery_time`, timestamps	N:1 com `users` (cliente e entregador) e `addresses`. 1:N com `order_items`.
`order_items`	`id`, `quantity`, `observation`, `meat_point`, `removed_ingredients`, `unit_price`, `addons_snapshot` (jsonb)	N:1 com `orders` (cascade) e `products`. N:N com `addons`.
`addresses`	`id`, `zipCode` (8 dígitos), `street`, `number`, `complement`, `neighborhood`, `city`, `state` (UF), `nickname`, `latitude`, `longitude`, `created_at`, `deleted_at` (exclusão lógica)	N:1 com `users`. Pedidos antigos continuam exibindo o endereço excluído.
`store_config`	`id`, `is_open`, `opening_message`, `closing_message`	Linha única (a de menor id), criada na subida com a loja aberta.

14. Pagamentos (sandbox e produção)

Como o modo é decidido

`resolvePaymentMode` (`backend/src/payment/payment-rules.ts`): `PAYMENT_SANDBOX=true/false` decide. Vazio = sandbox quando não há `MP_ACCESS_TOKEN` ou quando ele começa com `TEST-` , exceto com `NODE_ENV=production` (nesse caso, só com `PAYMENT_SANDBOX=true` explícito). O modo aparece no log da API na subida.

	Sandbox sem `MP_ACCESS_TOKEN`	**Sandbox com token `TEST-`**	**Produção**
Cartão	Aprovação simulada.	Enviado ao Mercado Pago. Recusa ou falha → aprovação simulada.	Resultado real. Recusas e falhas chegam ao cliente.
PIX	QR de teste (não pode ser pago), aprovado após `PIX_SANDBOX_APPROVAL_SECONDS` .	QR do Mercado Pago, aprovado automaticamente após o prazo. Falha → QR de teste.	Aprovado só quando o Mercado Pago confirma.
Sem token	Tudo simulado.		`503` "Pagamento online indisponível..." (pagar na entrega continua funcionando).

Respostas simuladas trazem `simulation: true` . O log registra "APROVADO POR SIMULAÇÃO" (cartão) ou "SIMULADO"/"SIMULAÇÃO de sandbox" (PIX).

Dados de teste

Número	**Validade**	**CVV**	**Titular**	**CPF**
`5031 4332 1540 6351`	`11/30`	`123`	`APRO`	`123.456.789-09`

PIX: gere o QR e aguarde cerca de 5 s. A web consulta o status a cada 3 s e o app a cada 4 s, e o pedido é registrado sozinho após a aprovação. O código PIX expira na tela depois de 10 minutos (é preciso gerar outro).

Fluxo e verificações

1. O front envia `order` (o mesmo corpo do orçamento) para `/payment/card` (com o token do cartão, bandeira e CPF/CNPJ) ou `/payment/pix` . O valor é sempre calculado pela API.
2. Com o pagamento aprovado, o front cria o pedido em `POST /orders` com `paymentId` e a forma de pagamento.
3. Antes de gravar, a API confere: pagamento criado por ela para o mesmo cliente, forma compatível (PIX ↔ PIX, cartão ↔ crédito/débito), aprovado (pagamentos reais são consultados de novo no Mercado Pago), valor ≥ total − R$ 0,01 e não usado em outro pedido (`payment_id` único). Qualquer falha → `400` , e nada é gravado.

4. Se a criação do pedido falhar depois de um pagamento aprovado, a web guarda o pagamento na aba para tentar de novo sem cobrar outra vez.

> **Limitação atual:** o registro dos pagamentos fica em memória na API (validade de 24 h). Um reinício entre o pagamento e a criação do pedido faz a criação falhar ("Pagamento não encontrado ou expirado..."), e o pagamento precisa ser conferido no painel do Mercado Pago. O webhook (`POST /payment/webhook`) só confirma o recebimento. Rode uma única instância da API, ou use afinidade de sessão. A confirmação por webhook está no roadmap.

---

15. Integrações externas

Integração	Onde está	Necessária?	Observações
Mercado Pago	API: `src/payment/` . Web: `src/services/mercadoPago.ts` e checkout. App: `screens/PaymentCard` (API REST de tokenização).	Para pagamento online real	Access token só na API (`MP_ACCESS_TOKEN`). Chaves públicas no web/app (padrão: chave de teste). Sem token, o sandbox simula.
football-data.org	`backend/src/promotions/promotions.service.ts`	Opcional	`FOOTBALL_DATA_TOKEN` . Prioriza o Brasileirão e usa cache. Qualquer falha cai na promoção do dia.
ViaCEP	`backend/src/addresses/addresses.service.ts`	Opcional	Primeira tentativa de busca de CEP (timeout de 5 s).
BrasilAPI (CEP)	`backend/src/addresses/addresses.service.ts`	Opcional	Reserva quando o ViaCEP falha. Sem os dois, o endereço é preenchido manualmente.
Serveo	`frontend-mobile/start-tunnel.js`	Só para o app fora da rede	Requer `ssh` e internet. Grava a URL em `.env.local` .
Google Maps	Quadro de pedidos (web e app), retirada no checkout (web)	Não	Apenas links de busca de endereço (sem chave de API).

---

16. Fluxos principais

Cadastro e login

1. O cliente se cadastra em `/signup` (web) ou em "Crie sua conta" (app), via `POST /users` , e vira `CLIENT` .
2. Contas da equipe são criadas pelo Administrador (web: **Equipe**; app: **Usuários**).
3. Login via `POST /auth/login` . O front guarda o token e o usuário e envia `Bearer` nas próximas chamadas.

Pedido com entrega

1. O cliente vê o cardápio (`GET /products`), o status da loja e o banner.
2. Monta o carrinho: quantidade, adicionais, ponto da carne, ingredientes removidos e observação.
3. No checkout, escolhe ou cadastra um endereço (com busca de CEP). A tela mostra o orçamento de `POST /orders/quote` .

4. Paga online (PIX ou cartão, e o pedido nasce `PAID`) ou escolhe pagar na entrega (dinheiro com troco ou maquininha).
5. O pedido entra como `PENDING` no quadro. Cozinha (ou gestão) aceita e marca como pronto.
6. A gestão despacha para um entregador, ou o entregador pega o pedido. O status vai para `DELIVERING`.
7. O entregador confirma a entrega (`DONE`). Se era pagamento na entrega, vira `PAID`.

Pedido para retirada

`TAKEOUT` não exige endereço, não tem taxa e tem tempo estimado de 15-20 min. Depois de pronto, aparece em "Retirar no Balcão", e a gestão marca "Entregue ao Cliente" (`READY_FOR_PICKUP → DONE`).

Logística

1. No pedido pronto, a gestão clica em **Despachar**. A API lista os entregadores (`/orders/drivers/list`) e sugere pedidos do mesmo bairro (`/orders/:id/nearby`).
2. `PATCH /orders/:id/assign` atribui o entregador e coloca o pedido em `DELIVERING`.
3. O entregador vê endereço, mapa, troco a levar e, no app, o botão de ligar para o cliente.

---

17. Uploads e imagens

- Produtos (`POST/PATCH /products`), avatares (`POST/PATCH /users`) e `POST /uploads` aceitam **JPG, PNG ou WebP de até 5 MB** no campo `file` (um arquivo por envio).
- O tipo é conferido pelo conteúdo (assinatura do arquivo), e não pela extensão. A imagem é rotacionada conforme o EXIF, limitada a 800 px de largura e convertida para WebP (qualidade 80) com Sharp.
- Os arquivos ficam em `UPLOADS_DIR` e são servidos em `/uploads/<arquivo>`. Produtos guardam só o nome do arquivo, e avatares guardam `/uploads/<arquivo>`.
- Um formulário inválido não deixa imagem órfã, e se a operação falhar o arquivo recém-gravado é apagado.
- `POST /maintenance/optimize-images` converte imagens antigas (não WebP) de produtos e avatares, mantendo os originais em disco.

```
POST /uploads            (Authorization: Bearer <token de ADMIN>)
Content-Type: multipart/form-data
Campo do arquivo: file
Resposta: { "filename": "compressed-<data>-<código>.webp" }
```

18. Testes

Unitários da API (Jest)

```
cd backend
npx jest            # ou npm test
npm run test:cov    # cobertura
```

Não precisam de banco. Cobrem regras de pedidos (dinheiro, preço, entrega, troco, perfis, transições, cancelamento e despacho), serviço de pedidos, pagamentos (sandbox e produção), autenticação, permissões de usuários, cardápio, uploads, loja, promoções, endereços e configuração.

E2E da API com Jest

```
DB_DATABASE=comestra_test npm run test:e2e                 # Linux (banco de teste já criado)
$env:DB_DATABASE="comestra_test"; npm run test:e2e         # Windows (PowerShell)
```

Suíte ponta a ponta (Playwright)

```
cd e2e
npm install
npm test            # recria o banco comestra_e2e, compila a API e roda tudo
npm run test:api    # só a API
npm run test:web    # só o navegador
npm run report      # relatório HTML
```

- Ambiente isolado: API de teste em `:3001` (banco `comestra_e2e`, imagens em `e2e/.uploads`, sandbox ligado) e web de teste em `:5174`. O Playwright sobe os servidores ou reaproveita os que já estão no ar.
- Se a API de teste já estiver rodando, o banco não é recriado e a API não é recompilada.
- Se o `postgres` tiver senha, informe `DB_PASSWORD` no ambiente. Requer Google Chrome e dependências instaladas em `backend/` e `frontend-web/`.
- Estrutura: `tests/global.setup.ts` (login de todos os perfis e loja aberta), `tests/api/identity` (saúde, login, usuários, senhas fora das respostas, endereços, loja, uploads, banner), `tests/api/commerce` (catálogo, orçamento, acesso e fluxo dos pedidos, pagamentos), `tests/web/client` (loja) e `tests/web/admin` (painel).

Verificações executadas para este guia (11/10/2026)

Verificação	Resultado	Observação
`backend: npx jest`	OK	15 suítes, 157 testes passando.
`GET /health` (API de teste, :3001)	OK	`{"status":"ok","name":"Comestra","version":"0.0.1"}`
`GET /api` (Swagger)	OK	HTTP 200.
`POST /auth/login` com os 5 usuários padrão	OK	Cada conta entrou com o perfil esperado (`ADMIN`, `EMPLOYEE`, `KITCHEN`, `COURIER`, `CLIENT`).

19. Produção

1. `NODE_ENV=production` e `JWT_SECRET` forte: `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`.
2. `SEED_DEMO_USERS=false` (padrão em produção), com as senhas de demonstração trocadas ou as contas excluídas.
3. `PAYMENT_SANDBOX=false`, `MP_ACCESS_TOKEN` de produção e chaves públicas reais (`VITE_MP_PUBLIC_KEY`, `EXPO_PUBLIC_MP_PUBLIC_KEY`).
4. `DB_SYNCHRONIZE=false` com migrations (crie-as antes, já que o repositório ainda não as traz).
5. `CORS_ORIGINS` com os domínios reais da loja e do painel.
6. HTTPS na frente da API e do site, com `VITE_API_URL` e `EXPO_PUBLIC_API_URL` em `https://`.
7. `UPLOADS_DIR` em volume persistente, com backup do banco e das imagens.
8. Uma única instância da API (ou afinidade de sessão) enquanto o registro de pagamentos estiver em memória.
9. Build: API com `npm run build` e `npm run start:prod`. Web com `npm run build`, servindo `dist/` com fallback para `index.html` (rotas do React Router).
10. Dados da loja revisados (seção 20) e marca verificada (seção 22).

20. Personalização por instalação

O quê	Onde
Nome exibido	`APP_NAME` (API), `VITE_APP_NAME` (web), `EXPO_PUBLIC_APP_NAME` (app).
Marca (nome padrão, slug, domínio, slogan, e-mail de suporte)	`src/config/brand.ts` em cada app. O slug também prefixa as chaves de armazenamento e define o banco padrão.
Logo e ícones	Web: `public/favicon.svg` e `src/components/BrandLogo` (mesma arte). App: `assets/` e `app.json`.
Endereço, telefone e horários (dados de demonstração)	Web: `STORE_INFO` em `src/config/brand.ts` e o endereço de retirada em `src/pages/Client/Cart/index.tsx`. App: `src/components/Footer`.
Taxas e tempos de entrega	`DELIVERY_ZONES` em `backend/src/orders/order-rules.ts`.
Promoções do dia da semana	`WEEKLY_PROMOTIONS` em `backend/src/promotions/promotions.service.ts`.
Mensagens de loja aberta/fechada	`PATCH /store/status` (`opening_message`, `closing_message`).

21. Migração a partir do TorresBurgers

- **Banco:** o padrão agora é `comestra`. Para manter os dados, use `DB_DATABASE=torresburgers` (e `DB_PASSWORD=0000`, se a instalação dependia da senha padrão antiga) ou copie o banco com `createdb -T torresburgers comestra`.
- **Sessões:** as chaves mudaram (web `torresburgers.*` → `comestra.*`; app `@TorresBurgers:*` → `@comestra:*`). Todos precisam entrar de novo, e o carrinho e o tema salvos voltam ao padrão.

- **Equipe:** `POST /users` sem token de `ADMIN` cria só clientes. Para recuperar o acesso de administrador, use `npm run seed`.
- **.env antigo:** `PORT`, `JWT_EXPIRES_IN` (antes fixo em 1 dia) e `MP_ACCESS_TOKEN` (antes fixo no código) passaram a valer. Rotas antes públicas (`/addons` de escrita, `/uploads`, `/payment/*`) agora exigem o perfil adequado.
- **App:** o `npm run tunnel` não altera mais `src/services/api.ts`. A URL vai para `.env.local`.

---

22. Limitações conhecidas e roadmap

Limitações atuais

- Sem recuperação de senha por e-mail: um Administrador define uma nova senha.
- Atualização por consulta periódica, sem WebSockets (quadro a cada 5 s, Meus Pedidos na web a cada 10 s, status da loja a cada 30 s na web e 60 s no app).
- Banner de promoções e cupons apenas informativos: não há desconto no checkout.
- Uma loja por instalação.
- Registro de pagamentos em memória, e o webhook do Mercado Pago só confirma o recebimento.
- Sem migrations versionadas.
- Dados da loja, taxas de entrega e promoções definidos no código.
- App: sem envio de fotos de produto ou de perfil (feito pela web).

Roadmap

- Recuperação de senha por e-mail.
- Status do pedido em tempo real via WebSockets.
- Notificações push.
- Cupons e promoções aplicados no checkout.
- Multi-loja / multi-tenant.
- Confirmação do pedido pelo webhook do Mercado Pago.

> **Marca:** antes do lançamento comercial, faça a busca no INPI (pePI) e o pedido de registro pelo e-Marcas nas classes 9, 35, 42 e 43. Registre também o domínio `comestra.com.br`.

---

23. Troubleshooting

A API não conecta no banco

- Confirme que o PostgreSQL está rodando. Windows: serviço `postgresql-x64-<versão>` em `services.msc`. Linux: `systemctl status postgresql`.
- Confirme que o banco existe (`psql -U postgres -l`) e revise `DB_HOST`, `DB_PORT`, `DB_USERNAME`, `DB_PASSWORD` e `DB_DATABASE`.
- No Linux, a conexão é TCP: o usuário precisa de senha (ou de uma regra `trust` no `pg_hba.conf`).

A API não sobe em produção

"JWT_SECRET não definido...": defina `JWT_SECRET` no ambiente. Com `NODE_ENV=production`, ele é obrigatório.

Os usuários padrão não entram

- Rode `cd backend && npm run seed` para recriar as contas e redefinir as senhas.
- Confira se `SEED_DEMO_USERS` não está `false` e se a API aponta para o banco certo.

A web não acessa a API

- Abra `http://localhost:3000/health`. Confira `VITE_API_URL` e reinicie o `npm run dev` depois de mudá-la.
- Com `CORS_ORIGINS` preenchido, inclua a origem do site (ex.: `http://localhost:5173`).

O app não acessa a API

- Celular e computador no mesmo Wi-Fi, porta 3000 liberada no firewall e API rodando.
- Emulador Android: use `EXPO_PUBLIC_API_URL=http://10.0.2.2:3000` se o IP da rede não funcionar.
- Fora da rede: `npm run tunnel` (precisa de `ssh`) e depois `npx expo start --tunnel --clear`. Se o Serveo cair, reinicie o túnel e o Expo.
- Depois de mudar qualquer `EXPO_PUBLIC_*`: `npx expo start --clear`.

Pedido recusado

- "A loja está fechada no momento.": abra a loja no painel (Dashboard ou Pedidos) ou via `PATCH /store/status` com `{ "is_open": true }`.
- "Somente clientes fazem pedidos e pagamentos...": entre com uma conta `CLIENT`.
- "Endereço obrigatório para delivery." / "Endereço não encontrado.": cadastre ou selecione um endereço do próprio cliente.
- Produto "indisponível" ou "não está mais no cardápio": reative o produto ou tire-o da lixeira.

Pagamento online

- `503` "Pagamento online indisponível...": configure `MP_ACCESS_TOKEN` ou ligue o sandbox (desenvolvimento).
- "Pagamento não encontrado ou expirado...": a API reiniciou entre o pagamento e o pedido. Gere um novo pagamento (no sandbox) ou confira no painel do Mercado Pago.
- Cartão não carrega na web: o SDK do Mercado Pago precisa de internet. Use PIX ou o pagamento na entrega.

Upload falha

- Envie um único arquivo no campo `file`, em JPG, PNG ou WebP de até 5 MB.
- Verifique a permissão de escrita em `UPLOADS_DIR` (padrão `backend/uploads`).

Testes E2E não sobem

- Informe `DB_PASSWORD` no ambiente (o reset do banco não lê o `backend/.env`) e confira se o Google Chrome está instalado.
- Portas 3001/5174 ocupadas por outra aplicação: libere-as ou use `E2E_API_PORT` / `E2E_WEB_PORT`.

Build web ou Expo reclama do Node

Instale o Node.js 22 LTS ou mais recente.